Object-Oriented Programming

Composition, Inheritance and Ad-hoc polymorphism

Filip Křikava

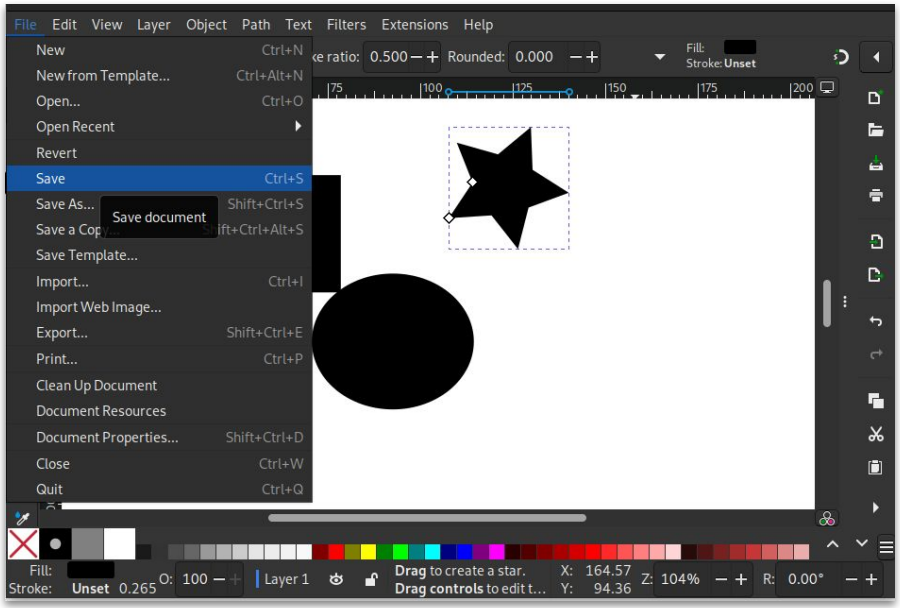
https://courses.fit.cvut.cz/BI-OOP/

Any questions?

Composition

Motivating example

- Imagine working on a vector graphics editor
- Next task is to add "save the canvas to disk" functionality



Motivating example

```
abstract class Shape:
  …


class Rectangle(w: Int, h: Int) extends Shape:
  …


class Canvas:
  val shapes = mutable.ArrayBuffer[Shape]()
  …


class CanvasSerializer:
  def serialize(canvas: Canvas): Array[Byte] =
    for s <- canvas.shapes do
      s match
        case Rectangle(w, h) => …

  …
```

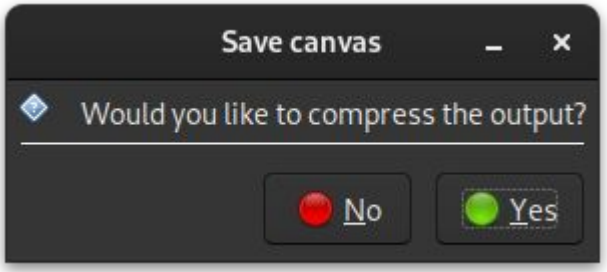
```
class SaveAction(serializer: CanvasSerializer):
  def run(canvas: Canvas, output: File) =
    val bytes = serializer.serialize(canvas)
    Files.write(output.toPath, bytes)



// wire it all from the menu
val canvas = …
val serializer = CanvasSerializer()
val saveAction = SaveAction(serializer)
saveAction.run(canvas, File("/tmp/test"))
```

Motivating example

- Add support for output compression



Motivating example

```
class CompressedCanvasSerializer(level: Int)
  extends CanvasSerializer:


  private def compress(bytes: Array[Byte]): Array[Byte] = ???


  override def serialize(canvas: Canvas): Array[Byte] =
    val bytes = super.serialize(canvas)
    compress(bytes)

  …
```

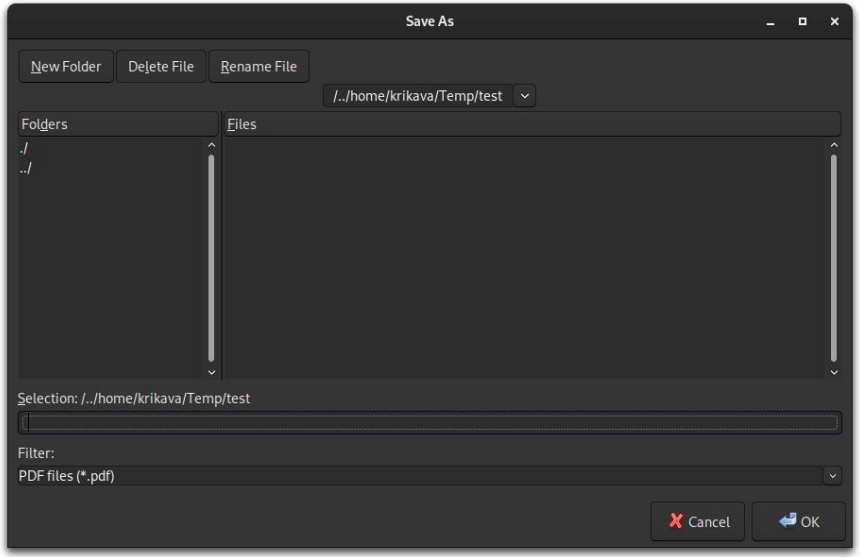
```
// no changes to the SaveAction
class SaveAction(serializer: CanvasSerializer):
  def run(canvas: Canvas, output: File) =
    val bytes = serializer.serialize(canvas)
    Files.write(output.toPath, bytes)


// the only change is the how we create a serializer
val canvas = …
val serializer = CompressedCanvasSerializer(1)

val saveAction = SaveAction(serializer)
saveAction.run(canvas, File("/tmp/test"))
```

Motivating example

- Add support for different formats
  - PDF
  - SVG
- But keep the compression possibility for both formats



Compression inherits one fixed format

The format implementations

```
class CanvasSerializer:
  def serialize(c: Canvas): Array[Byte] =
    encodeDefault(c)

class PDFCanvasSerializer
    extends CanvasSerializer:
  override def serialize(c: Canvas) =
    encodePDF(c)

class SVGCanvasSerializer
    extends CanvasSerializer:
  override def serialize(c: Canvas) =
    encodeSVG(c)
```

The existing compression subclass

```
class CompressedCanvasSerializer
    extends CanvasSerializer:

  override def serialize(c: Canvas) =
    compress(super.serialize(c))

val s = CompressedCanvasSerializer()
s.serialize(canvas)
// default -> compression
// It does not produce PDF or SVG.
```

The compressed serializer always uses the default format.

How do we get compressed PDF and compressed SVG?

Compression needs a subclass for each format

PDF family

```
class CompressedPDFCanvasSerializer
    extends PDFCanvasSerializer:

  override def serialize(c: Canvas) =
    compress(super.serialize(c))

// PDF -> compression
val pdf = CompressedPDFCanvasSerializer()
```

SVG family

```
class CompressedSVGCanvasSerializer
    extends SVGCanvasSerializer:

  override def serialize(c: Canvas) =
    compress(super.serialize(c))

// SVG -> compression
val svg = CompressedSVGCanvasSerializer()
```

The two overrides have the same body.

A third format needs another compression subclass.

All need to copy and paste the compress method from the `CompressedCanvasSerializer`

Use composition instead of inheritance

```
// 1. create an interface
abstract class CanvasSerializer:
  def serialize(c: Canvas): Array[Byte]


// 2. default serialization implementation
class DefaultCanvasSerializer extends CanvasSerializer:
  def serialize(c: Canvas) = encodeDefault(c)


// 3. add as many formats as you want
class PDFCanvasSerializer extends CanvasSerializer:
  def serialize(c: Canvas) = encodePDF(c)

class SVGCanvasSerializer extends CanvasSerializer:
  def serialize(c: Canvas) = encodeSVG(c)



// 4. use composition for stacking the behavior
class CompressedCanvasSerializer(next: CanvasSerializer)
  extends CanvasSerializer:

  def serialize(c: Canvas) = compress(next.serialize(c))
```

Create and use

```
def save(s: CanvasSerializer) =
  s.serialize(canvas)

DefaultCanvasSerializer()


PDFCanvasSerializer()

SVGCanvasSerializer()


CompressedCanvasSerializer(
  PDFCanvasSerializer())

CompressedCanvasSerializer(
  SVGCanvasSerializer())
```

Use composition instead of inheritance

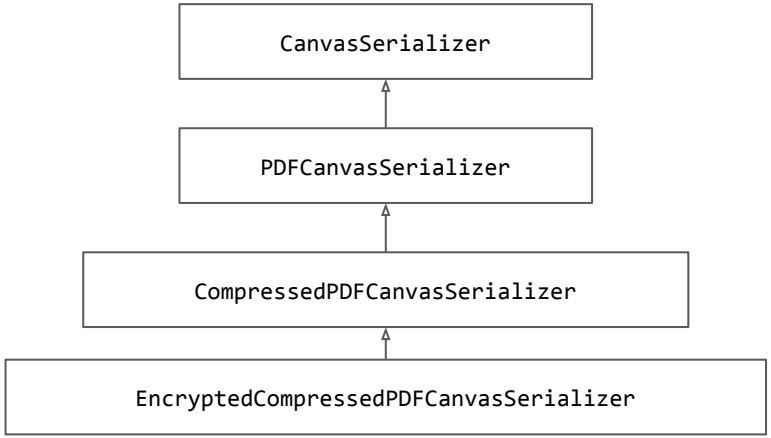
Bonus: it scales, adding encryption works with every format

```
class EncryptedCanvasSerializer(next: CanvasSerializer)
  extends CanvasSerializer:

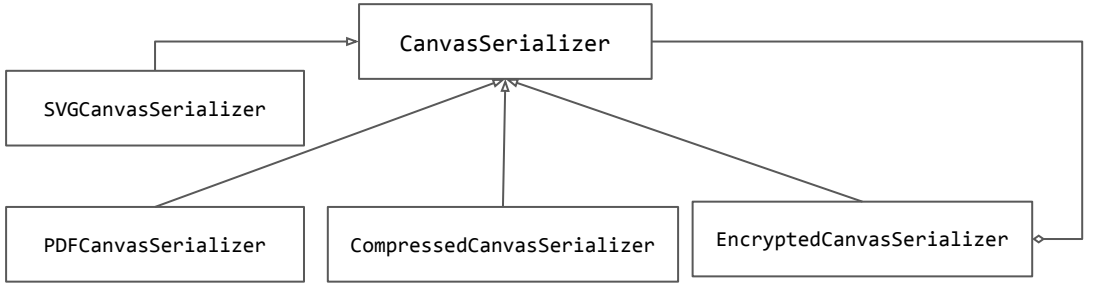
  def serialize(c: Canvas) = encrypt(next.serialize(c))


// PDF → compress → encrypt
val serializer =
  EncryptedCanvasSerializer(
    CompressedCanvasSerializer(
      PDFCanvasSerializer()
    )
  )
```

Use composition instead of inheritance



Inheritance

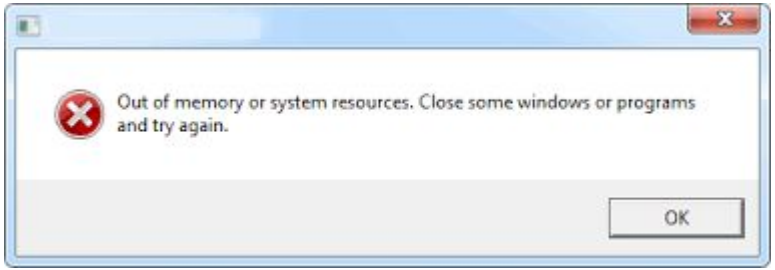


Composition

For other formats we will need to do duplicate both the compression and encryption.

Bonus Exercise

- With the `CanvasSerializer` you will soon start seeing out of memory exceptions
- Even more often when one uses compression
- Why is that?



Bonus Exercise

```
class SaveAction(serializer: CanvasSerializer):
  def run(canvas: Canvas, output: File) =
    // allocates big byte array!
    val bytes = serializer.serialize(canvas)
    // just to sink it in a file
    Files.write(output.toPath, bytes)


class CompressedCanvasSerializer(delegate: CanvasSerializer, level: Int) extends CanvasSerializer:
  override def serialize(canvas: Canvas): Array[Byte] =
    // allocates big byte array!
    val bytes = delegate.serialize(canvas)
    // compress again allocates so it can return the byte array
    compress(bytes)
```

Stream the output through a decorator

```
abstract class CanvasSerializer:
  def serialize(c: Canvas, out: OutputStream): Unit

class CompressedCanvasSerializer(next: CanvasSerializer) extends CanvasSerializer:
  def serialize(c: Canvas, out: OutputStream): Unit =
    val gzip = java.util.zip.GZIPOutputStream(out)
    next.serialize(c, gzip)
    gzip.finish()


class SaveAction(serializer: CanvasSerializer):
  def run(c: Canvas, file: java.io.File): Unit =
    scala.util.Using.resource(java.io.FileOutputStream(file)) { out =>
      serializer.serialize(c, out)
    }
```

Finish each compression wrapper. Close only the stream that you own.

Composition: design rules

- Define a small interface before you choose an implementation.
- If behaviors vary independently, put them in separate components.
- Use decorators to combine optional behavior without a subclass for each combination (decorator pattern).
- Choose and document the sequence of effects when you combine decorators.

In our serializer example:

```
Encrypted(Compressed(Pdf()))  // PDF → compress → encrypt
```

The reverse nesting encrypts first, then compresses.
Encrypted data usually compresses poorly.

Inheritance

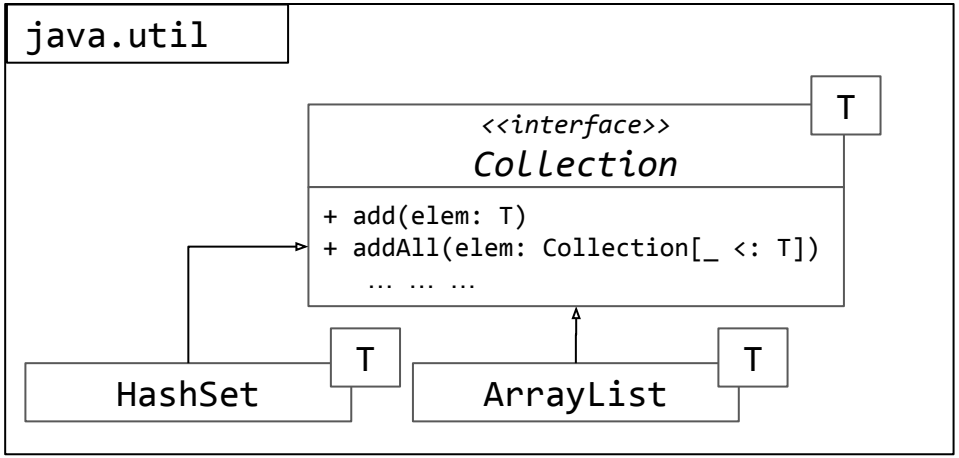
Inheritance

- Relates objects or classes together using is-a relationship
- Aims for code reuse and extension
- Allows a new object or a class to take structure and behavior of another object or class
- Enables polymorphism and code reuse

Two forms of inheritance

- Implementation inheritance reuses base code and gives the subclass the base type.
- A virtual call can reach an override in the subclass.
- Private base state still exists, but the subclass cannot access it directly.
- Interface inheritance specifies a shared contract without a shared base implementation.
- Every subtype must keep the promises of its interface.

Running example



- For the following examples we use Java collections
- `java.util.Collection` is an abstract mutable unordered collection
- `java.util.ArrayList` is a concrete implementation of a mutable indexed collection backed by an array
- `java.util.HashSet` is a concrete implementation of a mutable set back by a hash table

Add a new collection - a stack

```
class Stack[T] extends java.util.ArrayList[T]:
  def push(x: T): Unit = { add(x); () }
  def pop(): T = remove(size() - 1)

val s = Stack[String]()
s.push("A")
s.push("B")


s.add(0, "X")   // The stack API also exposes indexed insertion.
s.remove(1)     // A client can remove an arbitrary element.
```

Lesson: code reuse also exposes the base API.

This example shows an abstraction leak.

Hide the storage with composition

```
final class Stack[T]:
  private val elems = java.util.ArrayList[T]()

  def push(x: T): Unit = { elems.add(x); () }
  def pop(): T = elems.remove(elems.size() - 1)

val s = Stack[String]()
s.push("A")
s.push("B")
assert(s.pop() == "B")
// s.add(0, "X") does not compile.
```

Lesson: reuse the list privately.

Expose only operations that preserve the stack abstraction.

On the other hand the Stack is no longer part of the collection hierarchy.

Keep subtyping through a small interface

```
final class Stack[T] extends Collection[T]:
  private val elems = java.util.ArrayList[T]()

  def push(x: T): Unit = { elems.add(x); () }
  def pop(): T = elems.remove(elems.size() - 1)

  // all the Collection methods forward to delegate which itself is a Collection
  def size(): Int = elems.size()
  …
```

Lesson: combine interface inheritance with composition.

A counter depends on base self-calls

Add a count of submitted elements, including duplicates.

```
import java.util.{Collection, HashSet}

class CounterSet[T] extends HashSet[T]:
  private var attempts = 0
  def count: Int = attempts

  override def add(x: T): Boolean =
    attempts += 1
    super.add(x)

  override def addAll(xs: Collection[? <: T]): Boolean =
    attempts += xs.size()
    super.addAll(xs)
```

```
val c = CounterSet[Int]()

c.addAll(List(1, 2, 3))

c.size() // ???
c.count  // ???
```

A counter depends on base self-calls

Add a count of submitted elements, including duplicates.

```
import java.util.{Collection, HashSet}

class CounterSet[T] extends HashSet[T]:
  private var attempts = 0
  def count: Int = attempts

  override def add(x: T): Boolean =
    attempts += 1
    super.add(x)

  override def addAll(xs: Collection[? <: T]): Boolean =
    attempts += xs.size()
    super.addAll(xs)
```

```
val c = CounterSet[Int]()

c.addAll(List(1, 2, 3))

c.size() // 3
c.count  // ???
```

A counter depends on base self-calls

Add a count of submitted elements, including duplicates.

```
import java.util.{Collection, HashSet}

class CounterSet[T] extends HashSet[T]:
  private var attempts = 0
  def count: Int = attempts

  override def add(x: T): Boolean =
    attempts += 1
    super.add(x)

  override def addAll(xs: Collection[? <: T]): Boolean =
    attempts += xs.size()
    super.addAll(xs)
```

```
val c = CounterSet[Int]()

c.addAll(List(1, 2, 3))

c.size() // 3
c.count  // 6
```

Lesson: an inherited method can call your override.

A base implementation change can change subclass behavior.

Make sure, you fully understand the superclass behavior.

A wrapper controls the count boundary

```
import java.util.{Collection, HashSet, Set}

final class CounterSet[T] extends Set[T]:
  private val elems = HashSet[T]()
  private var attempts = 0
  def count: Int = attempts

  def add(x: T): Boolean =
    attempts += 1
    elems.add(x)

  def addAll(xs: Collection[? <: T]): Boolean =
    attempts += xs.size()
    elems.addAll(xs)

  def size: Int = elems.size

  …
```

Lesson: use composition and interface inheritance, delegate where reasonable

Composition can miss calls inside the delegate

```
trait Printer:
  def print(s: String): Unit
  def printTwice(s: String): Unit

class BasicPrinter extends Printer:
  def print(s: String): Unit = println(s)
  def printTwice(s: String): Unit =
    print(s)
    print(s)

class LoggedPrinter(next: Printer) extends Printer:
  def print(s: String): Unit =
    println("log")
    next.print(s)
  def printTwice(s: String): Unit = next.printTwice(s)

LoggedPrinter(BasicPrinter()).printTwice("Hi") // No log lines.
```

Lesson: a delegate keeps its own this. A forwarded bulk operation can bypass the wrapper policy.

Route the operation through the wrapper

```
class LoggedPrinter(next: Printer) extends Printer:
  def print(s: String): Unit =
    println("log")
    next.print(s)

  def printTwice(s: String): Unit =
    print(s)
    print(s)

LoggedPrinter(BasicPrinter()).printTwice("Hi") // Two log lines.
```

Lesson: apply the policy at every public entry point. Test bulk operations as well as single calls.

Composition and subtyping can work together

- An object uses another object as a component.
- The owner controls which operations it exposes.
- A wrapper can implement the same interface as its component.
- That design shares the contract without inheritance of the component implementation.

Inheritance can protect a stable lifecycle

```
abstract class TestCase:
  final def run(): Unit =
    setUp()
    try test()
    finally tearDown()

  protected def setUp(): Unit = ()
  protected def test(): Unit
  protected def tearDown(): Unit = ()

final class AdditionTest extends TestCase:
  protected def test(): Unit = assert(1 + 1 == 2)

AdditionTest().run()
```

Lesson: use inheritance for a stable framework algorithm with small, documented hooks. Keep the algorithm final.

Before you extend a class

- Write the base contract as observable behavior.
- Check that every base operation makes sense for the subtype.
- Check that the subtype accepts the same inputs and keeps the promised results.
- Use only documented hooks and protected APIs.
- If the base class does not support extension, use a component instead.

Inheritance: design rules

- If you only need code reuse, start with a private component.
- Use a small interface to keep useful subtyping.
- Choose a base class only if every subtype can keep its contract.
- Keep constructors free of calls to subclass overrides.
- Keep framework algorithms final and document their hooks.
- Test the public API and the calls between operations.

Multiple Inheritance

Problems with Inheritance

- Single inheritance
  - Simplest model, well accepted
  - Not expressive enough
    - to factor out all the common features shared by classes in a complex hierarchy ↦ forcing code duplication
    - e.g., you have `InputStream`, `OutputStream` and want `InputOutputStream`
  - Not allowing to share features inherited from different parents
- Multiple inheritance
  - Complicated model, has not achieved wide acceptance
  - Complicated implementation
  - Ambiguity problems (the diamond problem)
  - Accessing overridden features (one `super` is not enough in presence of multiple superclasses)

Role of a Class in OOP

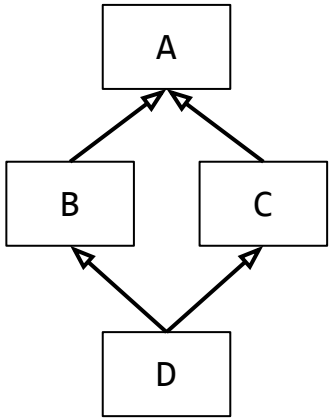
- A generator of instances
  - Provide a complete set of features
- A unit of reuse
  - Provide a minimal set of features that can be sensibly reused together
- A class[1] must have a fixed position in the class hierarchy which makes it difficult / impossible
  - to factor out wrapper methods
    - methods extending other methods with additional functionality as reusable classes
  - to resolve conflicting features inherited from different paths
  - to access or compose overridden features

[1] In a nominal statically typed world, but similar issues are in other typing schemes

Examples of approaches to multiple-inheritance

- Single inheritance with interfaces (Java, C#, Kotlin)
- Single inheritance with traits (Scala)
- Multiple inheritance (C++, Python)

The Diamond Problem



An ambiguity that arises:

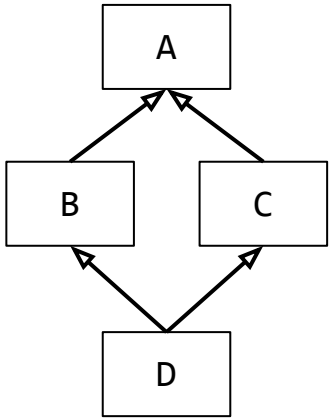
- when two classes B and C inherit from A, and
- class D inherits from both B and C.
- If there is a method in A

  that B and C have overridden,

  and D does not override,

  then which version of the method does D inherit: that of B, or that of C?

The Diamond Problem in C++



Non-virtual inheritance

```
struct A { int x = 0; };
struct B : A {};
struct C : A {};
struct D : B, C {};

D d;
// d.x = 1; // Ambiguous.
d.B::x = 1;
d.C::x = 2;
```

Two separate A subobjects.

Virtual inheritance

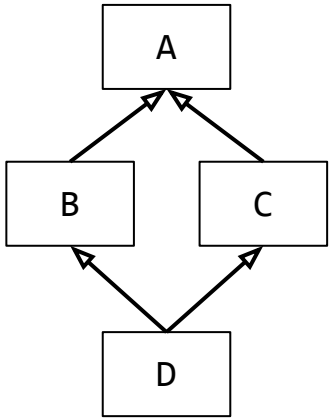
```
struct A { int x = 0; };
struct B : virtual A {};
struct C : virtual A {};
struct D : B, C {};

D d;
d.x = 1;
d.B::x = 2;
// d.C::x is now 2.
```

One shared A subobject.

If B and C override the same virtual method, D must resolve the conflict.

The Diamond Problem in Python




```
class A: pass

class B(A): pass

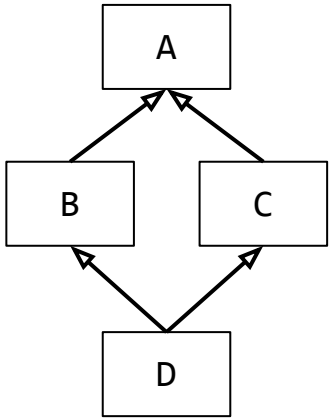
class C(A): pass

class D(B, C): pass
```

- Creates a list of classes using the C3 linearization:
  - children precede their parents, and
  - if a class inherits from multiple classes, they are kept in the order specified
- E.g.: the Method Resolution Order (MRO) is: D, B, C, A

```
- >>> D.__mro__
  (<class '__main__.D'>,
   <class '__main__.B'>,
   <class '__main__.C'>,
   <class '__main__.A'>,
   <type 'object'>)
```

The Diamond Problem in Java?



- Until Java 8 no problems
- Java 8 brought default methods
  - allowing one to add implementation to traits
  - this can cause diamond problem
  - must be resolved manually using explicit override of the conflicting method

The Diamond Problem in Java?

```java
interface A {
    default void m() {
        System.out.println("A.m");
    }
}

interface B extends A {
    default void m() {
        System.out.println("B.m");
    }
}

interface C extends A {
    default void m() {
        System.out.println("C.m");
    }
}

class D implements B, C {
}
```

```
error: types B and C are incompatible;
class D implements B, C {
^
  class D inherits unrelated defaults for m() from types B and C
1 error
```

```java
class D implements B, C {
    // The conflict has to be manually resolved
    @Override
    public void m() {
        B.super.m();
    }
}
```

Multiple inheritance: design rules

- Use several interfaces to express independent capabilities.
- Before you combine implementations, check the rules for method and state selection.
- Resolve conflicting methods explicitly.
- Do not assume that a diamond shares state in every language.
- If components need separate identities or lifetimes, compose separate objects.

Traits

Traits

- Fundamental units of reuse (that can be divorced from any class hierarchy)
- Contains method and field
  - Provide methods & fields
  - Require methods & fields (that are used but not defined in the trait)
- A class can mix in any number of traits and conflicts are resolved using overriding
- Classes retain primary roles as instance generators
- A trait is almost like an abstract class, but
  - cannot create instances
  - the calls to `super` are resolved dynamically (unlike in a class where it is static)
- Two common patterns
  - Widen thin interfaces
  - Stackable modifications

Widen thin interfaces

- Client/implementer trade-off between rich / thin interfaces
  - rich interfaces are convenient for clients (more methods to pick from)
  - thin interfaces are convenient for implementers (less code to write)

- For example, think about the difference between
  - using Iterator vs
  - creating a new Iterator implementation

Widen thin interfaces

```
class Rectangle(val topLeft: Point,
                val bottomRight: Point):
  def left = topLeft.x
  def right = bottomRight.x
  def top = topLeft.y
  def bottom = bottomRight.y
  def width = right - left
  def height = bottom - top
  ...
```

unrelated classes
(yet, with similar behavior)

```
abstract class UIComponent:
 def topLeft: Point
 def bottomRight: Point
 def left = topLeft.x
 def right = bottomRight.x
 def top = topLeft.y
 def bottom = bottomRight.y
 def width = right - left
 def height = bottom - top
 ...
```

```
trait Rectangular:
  def topLeft: Point
  def bottomRight: Point

  def left = topLeft.x
  def right = bottomRight.x
  def top = topLeft.y
  def bottom = bottomRight.y
  def width = right - left
  def height = bottom - top
```

required methods
(It can only be mixed in to a type that has these methods)

provided methods

```
class Rectangle(val topLeft: Point,
                val bottomRight: Point) extends Rectangular:
  ...
}

abstract class UIComponent extends Rectangular
  ...
}
```

trait mix in

Example of CharSequence

java.lang

Interface CharSequence

All Known Subinterfaces:
Name

All Known Implementing Classes:
CharBuffer, Segment, String, StringBuffer, StringBuilder

---

public interface **CharSequence**

A CharSequence is a readable sequence of char values. This interface provides uniform, read-only access to many different kinds of char sequences. A char value represents a character in the *Basic Multilingual Plane (BMP)* or a surrogate. Refer to Unicode Character Representation for details.

This interface does not refine the general contracts of the equals and hashCode methods. The result of comparing two objects that implement CharSequence is therefore, in general, undefined. Each object may be implemented by a different class, and there is no guarantee that each class will be capable of testing its instances for equality with those of the other. It is therefore inappropriate to use arbitrary CharSequence instances as elements in a set or as keys in a map.

Since:
1.4

Method Summary

All Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method and Description
char	**charAt**(int index) Returns the char value at the specified index.
default IntStream	**chars**() Returns a stream of int zero-extending the char values from this sequence.
default IntStream	**codePoints**() Returns a stream of code point values from this sequence.
int	**length**() Returns the length of this character sequence.
CharSequence	**subSequence**(int start, int end) Returns a CharSequence that is a subsequence of this sequence.
String	**toString**() Returns a string containing the characters in this sequence in the same order as this sequence.

https://docs.oracle.com/javase/8/docs/api/java/lang/CharSequence.html

https://www.scala-lang.org/api/3.5.2/scala/collection/StringOps.html

Exercise: widen a text sink interface

```
class ConsoleSink:
  def write(text: String): Unit = print(text)
  def writeLine(line: String): Unit = write(line + "\n")
  def writeLines(lines: Iterable[String]): Unit =
    lines.foreach(writeLine)

class BufferSink:
  val buffer = StringBuilder()
  def write(text: String): Unit = { buffer.append(text); () }
  def writeLine(line: String): Unit = write(line + "\n")
  def writeLines(lines: Iterable[String]): Unit =
    lines.foreach(writeLine)

val sink = BufferSink()
sink.writeLines(List("A", "B"))
```

Extract one required method. Put the two repeated operations in a trait.

One required method, two provided methods

```
trait TextSink:
  def write(text: String): Unit
  def writeLine(line: String): Unit = write(line + "\n")
  def writeLines(lines: Iterable[String]): Unit =
    lines.foreach(writeLine)

class ConsoleSink extends TextSink:
  def write(text: String): Unit = print(text)

class BufferSink extends TextSink:
  val buffer = StringBuilder()
  def write(text: String): Unit = { buffer.append(text); () }

val sink = BufferSink()
sink.writeLines(List("A", "B"))
sink.buffer.toString // "A\nB\n"
```

Implement write once for each destination. The trait supplies writeLine and writeLines.

Stackable pattern

- Traits can represent modifications (essentially a decorator)
- The can be stacked next to each other

Stackable pattern - example

Create a stack of integers

Stackable pattern - example

```
trait IntStack:
  def push(e: Int): IntStack
  def pop(): Int

class BasicIntStack extends IntStack:
  private var elems = List[Int]()

  def push(e: Int): IntStack =
    elems = e :: elems
    this

  def pop(): Int = elems match
    case x :: xs =>
      elems = xs
      x
    case Nil => throw new NoSuchElementException

  override def toString = elems.mkString("Stack(", ",", ")")
```

Stackable pattern - example

Create a stack of integers

Create a stack of integers that increments pushed integers

Create a stack of integers that doubles pushed integers

Create a stack of integers that filters out negative values

Create a stack of integers that can increment, double and filter out values

Stackable pattern - example

```
trait IntStack:
  def push(e: Int): IntStack
  def pop(): Int
```

This type can only be mixed into classes that mix in IntStack

```
trait Incrementing extends IntStack:
  abstract override def push(e: Int): IntStack = super.push(e + 1)

trait Doubling extends IntStack:
  abstract override def push(e: Int): IntStack = super.push(e * 2)

trait FilteringNegative extends IntStack:
  abstract override def push(e: Int): IntStack = if (e >= 0) super.push(e) else this
```

This trait must be mixed into some class that has a concrete definition of the method

https://scastie.scala-lang.org/9mqx2LCiQwafyqQiBNOKsg

Stackable pattern - example

```
trait IntStack
class BasicIntStack extends IntStack
trait Incrementing extends IntStack
trait Doubling extends IntStack
trait FilteringNegative extends IntStack


class MyStack extends BasicIntStack with Doubling with Incrementing with FilteringNegative
val s1 = new MyStack
val s2 = new BasicIntStack() with Incrementing with Doubling

trait Filtering(p: Int => Boolean) extends IntStack:
  abstract override def push(e: Int): IntStack = if (p(e)) super.push(e) else this

val s3 = new MyStack() with Filtering(x => (x & 1) == 0)
s3.push(1)
s3.push(2)
s3.push(3)
s3 // ???    6
```

Ad hoc mix in

Traits vs Multiple Inheritance

- In traits `super` uses late binding - dispatched dynamically
- Method call is determined by linearization

```
trait Incrementing extends IntStack {
  abstract override def push(e: Int): IntStack = super.push(e + 1)
}

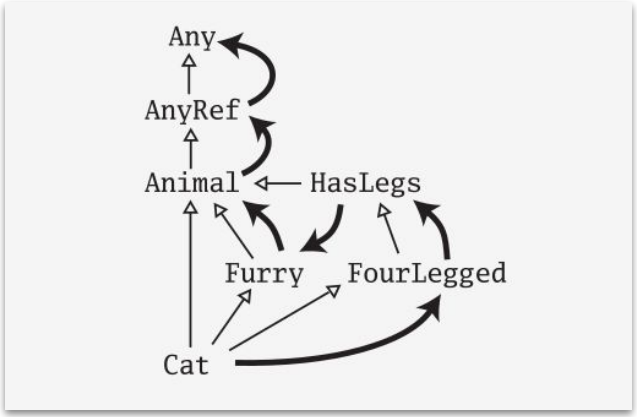
trait Doubling extends IntStack {
  abstract override def push(e: Int): IntStack = super.push(e * 2)
}


val s1 = new BasicIntStack() with Incrementing with Doubling
s1.push(2)
s1.pop()

val s2 = new BasicIntStack() with Doubling with Incrementing
s2.push(2)
s2.pop()
```

Traits Linearization

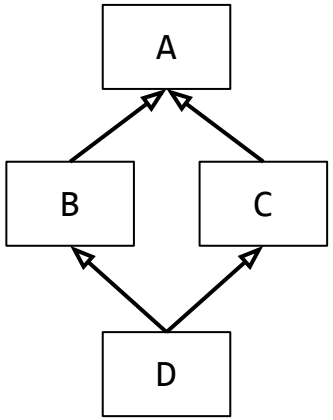
```
class Animal
trait Furry extends Animal
trait HasLegs extends Animal
trait FourLegged extends HasLegs
class Cat extends Animal with Furry with FourLegged
```



- Method are resolved using a right-first depth-first search of extended 'traits', before eliminating all but the last occurrence of each module in the resulting list

```
Animal:     Animal -> AnyRef -> Any
Furry:      Furry -> Animal -> AnyRef -> Any
FourLegged: FourLegged -> HasLegs -> Animal -> AnyRef -> Any
HasLegs:    HasLegs -> Animal -> AnyRef -> Any
Cat:        Cat -> FourLegged -> HasLegs -> Furry -> Animal -> AnyRef -> Any
```

The Diamond Problem in Scala?



- Scala allows multiple inheritance through traits
- It resolves method through linearization using right-first, depth-first algorithm, no diamond problem

```
trait A:
  def m(): Unit = println("A.m()")

trait B extends A:
    override def m(): Unit = println("B.m()")

trait C extends A:
    override def m(): Unit = println("C.m()")

class D extends B, C

def main(args: Array[String]): Unit =
    val d = D()
    d.m() // ???
```

The Diamond Problem in Scala?

```scala
trait B:
  def m(): Unit = println("B.m()")

trait C:
  def m(): Unit = println("C.m()")

class D extends B, C

def main(args: Array[String]): Unit =
    val d = D()
    d.m() // ???
```

- No diamond

```
[error] error overriding method m in trait B of type (): Unit;
[error]   method m in trait C of type (): Unit class D inherits
conflicting members:
[error]   method m in trait B of type (): Unit  and
[error]   method m in trait C of type (): Unit
[error] (Note: this can be resolved by declaring an override in class D.)
[error] class D extends B,C
[error]       ^
```

The Diamond Problem in Scala?

```
trait B:
  def m(): Unit = println("B.m()")

trait C:
  def m(): Unit = println("C.m()")

class D extends B, C:
  override def m(): Unit = super.m()

def main(args: Array[String]): Unit =
    val d = D()
    d.m() // C.m()
```

- No diamond
- Still there can be a conflict that needs an explicit resolution

The Diamond Problem in Scala?

```
trait B:
  def m(): Unit = println("B.m()")

trait C:
  def m(): Unit = println("C.m()")

class D extends B, C:
  override def m(): Unit = super[B].m()

def main(args: Array[String]): Unit =
    val d = D()
    d.m() // B.m()
```

- No diamond
- Still there can be a conflict that needs an explicit resolution
- Scala supports qualified `super`

To Trait or not to Trait

- If the behavior will not be reused: a concrete class
- If it might be reused in multiple, unrelated classes: a trait
- If you want to inherit from it in Java code: an abstract class
- If you still do not know: a trait (keeps more options open)

One operation follows one super chain

- The rightmost modifier runs first.
- Each modifier calls the next implementation through super.

```
val a = new BasicIntStack with Incrementing with Doubling
a.push(2)
a.pop() // 5: first double 2, then add 1.


val b = new BasicIntStack with Doubling with Incrementing
b.push(2)
b.pop() // 6: first add 1, then double 3.
```

Use one cooperative super chain. Do not call two parent methods to imitate composition.

Traits: design rules

- Use a trait for a small capability that unrelated classes can share.
- Keep required methods explicit and derive convenience methods from them.
- For a stackable modification, call super once for each accepted operation.
- Trace the rightmost trait first when you reason about a mixed class.
- Use an explicit override to resolve an unrelated method conflict.
- If behavior must change at run time, use replaceable components.

Ad-hoc polymorphism

Ad-hoc polymorphism

- One operation has different implementations for different types.
- The types do not need a common superclass.
- We keep familiar syntax and choose behavior that fits each type.
- Examples: integer addition, floating-point addition, and string concatenation.
- Overloads and type classes provide two forms of ad-hoc polymorphism.

Rational: a type that feels built in

```
val half = Rational(1, 2)

half + Rational(1, 3)  // 5/6
half + 2               // 5/2

2 * half               // 1/1
```

The result is a Rational. The class uses BigInt and reduces each fraction.

Operator overloads: Rational op Int / Double

```
final class Rational private (val n: BigInt, val d: BigInt):

  def +(r: Rational): Rational = Rational(n * r.d + r.n * d, d * r.d)
  def *(r: Rational): Rational = Rational(n * r.n, d * r.d)

  def +(x: Int): Rational = this + Rational(x, 1)
  def *(x: Int): Rational = this * Rational(x, 1)


val half = Rational(1, 2)
half + 2  // The compiler selects +(Int).
```

Operators are methods. The same pattern applies to subtraction and division.

The left operand selects the receiver

```
2 * half
// Int has no applicable *(Rational) overload.

Rational(2, 1) * half   // 1/1
```

We cannot add overloads to Int or Double. An implicit conversion can adapt the receiver.

Implicit conversions in Scala 3

```
import scala.language.implicitConversions

object RationalConversions:
  given intToRational: Conversion[Int, Rational] with
    def apply(x: Int): Rational = Rational(x, 1)

import RationalConversions.given

2 * half    // intToRational(2) * half
```

The Double conversion uses its shortest decimal form. It accepts only finite values.

How an implicit conversion works

- The compiler first tries the expression with its original types.
- If the expected type or a member call does not fit, it searches for an applicable conversion.
- The compiler inserts a conversion call. The program executes that call at run time.
- Visible definitions and the relevant companion objects supply candidates.
- A missing conversion or an unresolved ambiguity causes a compile error.
- A conversion adapts a value.

Extend Rational with comparison operators

```
final class Rational private (val n: BigInt, val d: BigInt):
  // Keep the existing construction and arithmetic.


val half = Rational(1, 2)
half < Rational(2, 3)   // true
half >= Rational(2, 4)  // true


  def <(r: Rational): Boolean = n * r.d < r.n * d
  def >(r: Rational): Boolean = r < this
  def <=(r: Rational): Boolean = !(this > r)
  def >=(r: Rational): Boolean = !(this < r)
```

Can a trait derive these four operators from one comparison method?

Rational: comparison with Ordered

```
final class Rational private (val n: BigInt, val d: BigInt)
    extends Ordered[Rational]:
  def compare(r: Rational): Int =
    (n * r.d).compare(r.n * d)
```

One compare method supplies <, >, <=, and >=.

```
trait Ordered[A] extends Any with java.lang.Comparable[A]:

 def compare(that: A): Int

  def <  (that: A): Boolean = (this compare that) <  0

  def >  (that: A): Boolean = (this compare that) >  0

  def <= (that: A): Boolean = (this compare that) <= 0

  def >= (that: A): Boolean = (this compare that) >= 0

  def compareTo(that: A): Int = compare(that)
```

required methods
(It can only be mixed in to a type that has these methods)

provided methods

Ad-hoc polymorphism: design rules

- Use familiar operators only if their meaning stays clear.
- Define overloads for the operand types that your API supports.
- State the result type and the numeric precision policy.
- Keep implicit conversions small and predictable.
- If a conversion can lose important information, make the conversion explicit.
- Use an extension method if you only need new method syntax.
- If a trait supplies related operators, implement its primitive operation.

Type classes

Exercise: save a Player as JSON

```
case class Player(name: String, score: Seq[Int])

val alice = Player("Alice", Seq(1, 2))
val bob = Player("Bob", Seq(3, 4))
val players: Seq[Player] = Seq(alice, bob)

// The JSON for alice:
// {"name":"Alice","score":[1,2]}

// The calls that we want:
save(alice)
save(players)
```

Keep the data model separate from the JSON format. Support sequences without repeated code.

First attempt: use interface inheritance

```
trait Jsonable:
  def toJSON: String

def save(x: Jsonable): Unit = println(x.toJSON)

case class Player(name: String, score: Seq[Int]) extends Jsonable:
  def toJSON: String =
    val scores = score.mkString("[", ",", "]")
    s"""{"name":${quote(name)},"score":$scores}"""

save(Player("Alice", Seq(1, 2)))
```

Player now depends on JSON. We must change its source to add this behavior.

An adapter removes the source dependency

```
def playerJsonable(p: Player): Jsonable =
  new Jsonable:
    def toJSON: String =
      val scores = p.score.mkString("[", ",", "]")
      s"""{"name":${quote(p.name)},"score":$scores}"""

save(playerJsonable(alice))
```

The adapter wraps each value. Could one reusable object describe JSON for the whole Player type?

Extract the behavior: Jsonable[T]

```
trait Jsonable[T]:
  def toJSON(x: T): String

def save[T](x: T, format: Jsonable[T]): Unit =
  println(format.toJSON(x))

// Player does not extend Jsonable.
case class Player(name: String, score: Seq[Int])
```

Jsonable[T] describes a capability of T. An instance holds the implementation for that type.

Define an instance for Player

```
val playerJson: Jsonable[Player] = new Jsonable[Player]:
  def toJSON(p: Player): String =
    val scores = p.score.mkString("[", ",", "]")
    s"""{"name":${quote(p.name)},"score":$scores}"""

save(alice, playerJson)
save(bob, playerJson)

// A single-method trait also accepts a function.
val intJson: Jsonable[Int] = x => x.toString
val stringJson: Jsonable[String] = x => quote(x)
```

We reuse the same behavior object for many values. The Player class stays unchanged.

Compose instances for sequences

```
def seqJsonable[T](item: Jsonable[T]): Jsonable[Seq[T]] =
  new Jsonable[Seq[T]]:
    def toJSON(xs: Seq[T]): String =
      xs.map(item.toJSON).mkString("[", ",", "]")

val playersJson = seqJsonable(playerJson)
save(players, playersJson)

val teamsJson = seqJsonable(playersJson)
save(Seq(players), teamsJson)
```

An instance for T gives us an instance for Seq[T]. The same rule supports nested sequences.

Let the compiler supply the instance

```
given playerFormat: Jsonable[Player] = playerJson
given stringFormat: Jsonable[String] = stringJson

def save[T](x: T)(using format: Jsonable[T]): Unit =
  println(format.toJSON(x))

save(alice)

// The same call with an explicit context argument:
save(alice)(using playerFormat)
```

given supplies a candidate. using requests an argument of that type.

Conditional instances compose automatically

```
given seqJson[T](using item: Jsonable[T]): Jsonable[Seq[T]] =
  seqJsonable(item)

save(players)
save(Seq(players))

// The compiler supplies the arguments for this call:
save[Seq[Player]](players)(using
  seqJson[Player](using playerFormat)
)
```

The compiler applies the sequence rule recursively until it finds an instance for each element type.

Follow the search for an instance

```
save(Seq(players))

// The compiler needs:
Jsonable[Seq[Seq[Player]]]
  seqJson[Seq[Player]]
    Jsonable[Seq[Player]]
      seqJson[Player]
        Jsonable[Player]
          playerFormat

// The resulting argument expression:
seqJson[Seq[Player]](using
  seqJson[Player](using playerFormat)
)
```

The search follows types at compile time. JSON conversion takes place at run time.

The type class pattern

- The type class defines the operations: Jsonable[T].
- An instance implements those operations for a type: Jsonable[Player].
- A generic function requests the capability: using Jsonable[T].
- The compiler selects the instance from the static types and the context.
- The value keeps its original type. No conversion to Jsonable takes place.
- We can add a type or an operation without a common domain superclass.

Scope, imports, and errors

```
object JsonFormats:
  given playerFormat: Jsonable[Player] = playerJson

import JsonFormats.given
save(alice)

// An explicit choice always remains possible.
save(alice)(using playerJson)
```

Use visible givens or companion objects. If no best candidate exists, compilation fails.

Context bounds and summon

```scala
def save[T: Jsonable](x: T): Unit =
  val format = summon[Jsonable[T]]
  println(format.toJSON(x))

// The context bound is shorthand for:
def save[T](x: T)(using format: Jsonable[T]): Unit =
  println(format.toJSON(x))

// Scala 2 names for the same pattern:
// (implicit format: Jsonable[T]) instead of (using ...)
// implicitly[Jsonable[T]] instead of summon[Jsonable[T]]
```

The bound requests evidence. It does not state that T extends Jsonable[T].

Restore method syntax with an extension

```
extension [T](x: T)
  def toJSON(using format: Jsonable[T]): String =
    format.toJSON(x)

alice.toJSON
players.toJSON

// Equivalent explicit calls to the behavior object:
playerFormat.toJSON(alice)
seqJson[Player](using playerFormat).toJSON(players)
```

The syntax feels like a member method. The Player class and the JSON behavior stay separate.

A standard type class: Ordering[T]

```
given byScore: Ordering[Player] =
  Ordering.by[Player, Long](_.score.map(_.toLong).sum)

players.sorted  // Alice, Bob
players.max     // Bob

val byName: Ordering[Player] = Ordering.by(_.name)
players.sorted(using byName.reverse) // Bob, Alice
```

Ordering[Player] stays outside Player. We can select a different order for each use.

Haskell: type classes are a language feature

```haskell
class Jsonable a where
  toJSON :: a -> String

save :: Jsonable a => a -> IO ()
save x = putStrLn (toJSON x)

-- Standard capabilities include Eq, Ord, Show, and Num.
-- (+) uses the Num capability.
```

A constraint requests an instance. Haskell uses class and instance declarations for this pattern.

Rust: traits and implementations

```
trait Jsonable {
    fn to_json(&self) -> String;
}

fn save<T: Jsonable>(x: &T) {
    println!("{}", x.to_json());
}

// impl Jsonable for Player supplies the behavior.
// Display, Ord, and Add are standard trait examples.
```

Trait bounds express capabilities. The orphan rule restricts implementations for external types.

Swift: protocols and extensions

```
protocol Jsonable {
    func toJSON() -> String
}

func save<T: Jsonable>(_ x: T) {
    print(x.toJSON())
}

// extension Player: Jsonable adds the conformance.
// Equatable, Comparable, and Codable are common examples.
```

A protocol constraint requests a capability. An extension can add conformance to an existing type.

Related mechanisms in other languages

- Java: Comparator<T> is a separate behavior object. The caller passes it explicitly.
- Python: __add__ and __radd__ support arithmetic and the reverse operand order.
- Python selects special methods at run time. It does not use Scala given search.
- These mechanisms share the goal: use one operation with behavior that fits each type.
- The rules for selection, extension, and conflicts differ between languages.

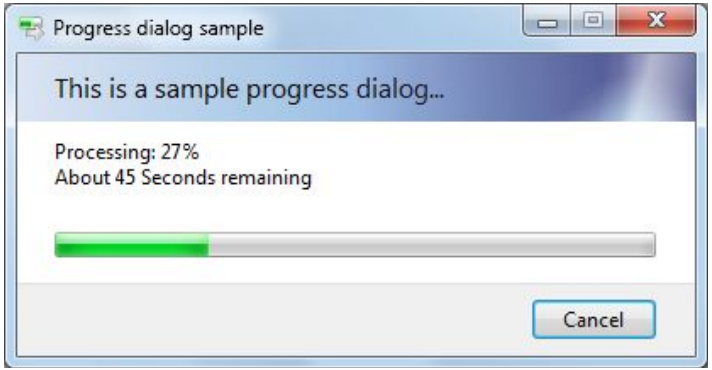
Type classes: design rules

- Keep domain data separate from independent capabilities.
- Put the capability in a small parameterized trait.
- Define clear default instances in companion objects or a deliberate import scope.
- Compose container instances from element instances.
- Use an explicit using argument if a call needs a different policy.
- Test the instance laws and the generated data, not only the type signatures.

Bonus

Bonus Exercise 2

- Saving could take a lot of time
- Add a progress bar to visualize the progress



Bonus Exercise 2

```
trait ProgressMonitor:
  def begin(totalWork: Int): Unit
  def progress(worked: Int): Unit
  def done(): Unit


class PDFCanvasSerializer extends CanvasSerializer:
  def serialize(canvas: Canvas, output: OutputStream, monitor: ProgressMonitor): Unit =
    monitor.begin(canvas.size)

    canvas.shapes.foreach { s =>
      serializeOne(s, output)
      monitor.progress(1)
    }

    monitor.done()

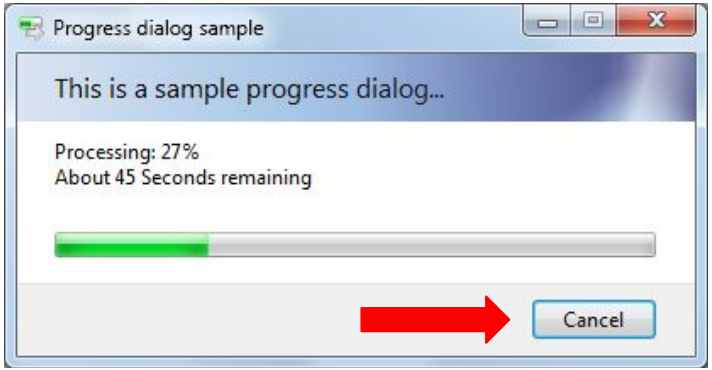
  def serializeOne(shape: Shape, output: OutputStream): Unit


class NullProgressMonitor extends ProgressMonitor:
  def begin(totalWork: Int): Unit = ()
  def progress(worked: Int): Unit = ()
  def done(): Unit

class PdfCanvasSerializer:
  def serialize(canvas: Canvas, output: OutputStream, monitor: ProgressMonitor = NullProgressMonitor): Unit = …
```

Bonus Exercise 2

- Saving could take a lot of time
- Add a progress bar to visualize the progress



- What about the cancel button?

Bonus Exercise 2

```
trait ProgressMonitor:
  def begin(totalWork: Int): Unit
  def progress(worked: Int): Unit
  def done(): Unit
  def isCancelled: Boolean


class PDFCanvasSerializer extends CanvasSerializer:
  def serialize(canvas: Canvas, output: OutputStream, monitor: ProgressMonitor): Unit =
    monitor.begin(canvas.size)

    canvas.shapes.takeWhile(_ => !monitor.isCancelled).foreach{ s =>
      serializeOne(s, output)
      monitor.progress(1)
    }

    monitor.done()

  def serializeOne(shape: Shape, output: OutputStream): Unit
```